Module 11

Polars for High-Performance Data Processing

Python Data Science Training — 3 Hours

Topics Covered:

11.1 Introduction to Polars — Design Philosophy & Benchmarks
11.2 Core Concepts — DataFrames, Expression API, Contexts
11.3 Data I/O ★ — CSV, JSON, Parquet, Avro, IPC, Excel
11.4 Data Manipulation — Filtering, Grouping, Window Functions
11.5 Data Cleaning — Nulls, Outliers, Type Casting
11.6 Joining and Combining — All Join Types, Concat
11.7 Lab — 1M+ Row Real-World Dataset with Performance Comparison

This document serves as both Trainer Slides and Post-Training Study Reference

11.1 Introduction to Polars

Polars is a blazing-fast DataFrame library written in Rust, with Python and Rust APIs. It was created by Ritchie Vink in 2020 and has rapidly become the go-to choice for teams that need to process large datasets quickly — often 5x to 50x faster than Pandas on equivalent operations.

Why Polars? Core Design Philosophy

Polars is built around four core principles:

- Vectorized execution — all operations use Apache Arrow columnar memory format, enabling SIMD CPU instructions
- Parallelism — computations are automatically parallelised across all CPU cores with no extra configuration
- Lazy evaluation — a query optimizer (similar to SQL query planners) can reorder and prune operations before execution
- Memory efficiency — Arrow-based zero-copy memory means less duplication and better cache utilization

Polars does NOT use NumPy internally. It sits directly on Apache Arrow, which means it can share memory with other Arrow-compatible tools (DuckDB, PyArrow, Spark, etc.) with zero copying.

Polars vs Pandas — When to Choose Each

Criterion	Polars	Pandas
Speed (large data)	Excellent — uses all CPU cores	Slower — mostly single-threaded
Memory usage	Lower — Arrow columnar format	Higher — NumPy row-oriented
API style	Expression-based, chainable	Imperative, index-centric
Lazy execution	Yes — built-in query optimizer	No (use Dask for lazy)
Ecosystem maturity	Growing rapidly (2020+)	Mature, very large ecosystem
ML integrations	Via Arrow exchange protocol	Direct sklearn, seaborn, etc.
Best for	Large files, ETL pipelines	EDA, small-medium datasets

Benchmark — Polars vs Pandas on 1M+ Rows

The following benchmark illustrates typical speedups. Results vary by hardware and operation type, but Polars consistently wins on aggregation, join, and group operations at scale.

Operation	Pandas (seconds)	Polars (seconds)	Speedup
Read CSV (1M rows)	2.1	0.35	~6x faster
GroupBy + Aggregation	3.8	0.28	~13x faster
Filter + Sort	1.2	0.18	~7x faster
Join (two 500K tables)	4.5	0.55	~8x faster
String operations	5.2	0.42	~12x faster
Write Parquet (1M rows)	1.8	0.22	~8x faster

Benchmarks are approximate. Polars advantage grows with dataset size. For datasets under 100K rows, Pandas may be equally fast due to parallelism overhead.

Installation and Setup

```
# Install Polars
pip install polars

# Install with all optional extras (Excel, timezone, plot support)
pip install 'polars[all]'

# Specific extras
pip install 'polars[xlsx2csv]'          # Excel reading
pip install 'polars[connectorx]'        # SQL database reading
pip install 'polars[pyarrow]'           # PyArrow integration

# Verify installation
import polars as pl
print(pl.__version__)                   # e.g., 0.20.x
print(pl.threadpool_size())             # Number of CPU threads Polars uses
```

Lazy vs Eager Execution — When to Use Each

This is the most important conceptual distinction in Polars.

	Eager Execution	Lazy Execution
How it works	Runs immediately, returns DataFrame	Builds a query plan, runs only on .collect()
API	pl.read_csv(), df.filter(), df.group_by()	pl.scan_csv(), lf.filter(), lf.group_by()
Memory	Full data in memory	Reads only what is needed (projection/predicate pushdown)
When to use	Interactive EDA, small data, debugging	Large files, pipelines, production ETL
Optimization	None	Automatic query plan optimization
Output	DataFrame	LazyFrame (call .collect() to execute)

```
import polars as pl

# Eager — runs each line immediately
df = pl.read_csv('sales.csv')                    # Loads entire file into memory
result = df.filter(pl.col('amount') > 100)       # Filters in memory

# Lazy — builds a query plan
lf = pl.scan_csv('sales.csv')                    # Opens file, does NOT load it
result = (
    lf
    .filter(pl.col('amount') > 100)              # Added to plan
    .select(['id', 'amount', 'date'])             # Added to plan
    .collect()                                    # NOW executes — reads only needed
cols/rows
)

# See the query plan before executing
```

```
print(lf.filter(pl.col('amount') > 100).explain())
```

Always use pl.scan_csv() / pl.scan_parquet() (lazy) for files larger than a few hundred MB. Polars will push filter and column selection down to the file reader, drastically reducing memory usage.

11.2 Core Concepts

DataFrames and Series in Polars

Like Pandas, Polars has DataFrame (2D table) and Series (1D column) as its primary data structures. The key difference is that Polars DataFrames have NO row index — every row is identified purely by its position. This is a deliberate design choice that eliminates a major source of confusion in Pandas.

```
import polars as pl

# Creating a DataFrame
df = pl.DataFrame({
    'name'   : ['Alice', 'Bob', 'Charlie', 'Dave'],
    'age'    : [25, 30, 35, 28],
    'salary' : [50000.0, 62000.0, 75000.0, 58000.0],
    'active' : [True, True, False, True],
})

print(df)
# shape: (4, 4)
# name     | age | salary  | active
# Alice    | 25  | 50000.0 | true
# ...

# Creating a Series
s = pl.Series('scores', [90, 85, 78, 92])
print(s.dtype)    # Int64
print(s.len())    # 4

# Key DataFrame attributes
df.shape          # (4, 4)
df.columns        # ['name', 'age', 'salary', 'active']
df.dtypes         # [Utf8, Int64, Float64, Boolean]
df.schema         # {'name': Utf8, 'age': Int64, ...}
df.height         # 4 (rows)
df.width          # 4 (columns)
```

The Expression API — Composable, Chainable Expressions

The Expression API is what makes Polars truly powerful. Instead of operating on columns directly, you build expressions using pl.col() and chain operations. These expressions are lazy by nature — they describe what to do, not when to do it.

The fundamental building block is pl.col('column_name'), which creates an expression referencing a column.

```
import polars as pl

# pl.col() is the entry point to expressions
expr = pl.col('salary') * 1.1          # A 10% raise expression (not computed yet)

# Expressions are applied inside contexts
df.with_columns(expr.alias('new_salary'))  # Now it is computed

# Chain multiple operations on a single expression
pl.col('name').str.to_uppercase().str.slice(0, 3)   # First 3 chars, uppercase
```

```
# Arithmetic between columns
pl.col('salary') - pl.col('salary').mean()          # Deviation from mean

# Conditional logic
pl.when(pl.col('age') >= 30)                         # if age >= 30
  .then(pl.lit('Senior'))                            # then 'Senior'
  .otherwise(pl.lit('Junior'))                      # else 'Junior'
  .alias('level')

# Aggregation expressions
pl.col('salary').mean()                              # Mean of salary column
pl.col('salary').sum() / pl.col('salary').count()   # Custom mean

# Select all columns
pl.all()                    # All columns
pl.all().exclude('name')    # All except 'name'

# Type-based selection
pl.col(pl.Float64)          # All Float64 columns
pl.col(pl.Utf8)             # All string columns
```

pl.col() never accesses data immediately. It is a description of a transformation. This is why Polars can optimize across many expressions at once before executing.

Contexts — select(), filter(), with_columns(), agg()

A context is a DataFrame method that accepts and evaluates expressions. Different contexts have different meanings for the same expression.

Context	Method	Returns	Use Case
Select	.select()	New DataFrame with selected/computed cols	Choose or transform columns
Filter	.filter()	Subset of rows	Row-level conditions
Add/Modify	.with_columns()	Same shape with added/updated columns	Add computed columns
Aggregate	.group_by().agg()	One row per group	Summaries by category

```
import polars as pl
df = pl.DataFrame({
    'region': ['North','South','North','East','South'],
    'product': ['A','B','A','C','A'],
    'sales'  : [100, 200, 150, 300, 250],
    'units'  : [10, 20, 15, 30, 25],
})

# select() — pick and compute columns (drops others)
df.select([
    pl.col('region'),
    pl.col('sales'),
    (pl.col('sales') / pl.col('units')).alias('avg_price'),
])

# filter() — keep rows matching condition
df.filter(pl.col('sales') > 150)

# with_columns() — add/replace columns, keeps all original cols
df.with_columns([
```

```
    (pl.col('sales') * 1.1).alias('sales_adjusted'),
    pl.col('product').str.to_uppercase().alias('product_upper'),
])

# group_by().agg() — aggregate per group
df.group_by('region').agg([
    pl.col('sales').sum().alias('total_sales'),
    pl.col('sales').mean().alias('avg_sales'),
    pl.col('units').sum().alias('total_units'),
    pl.col('product').n_unique().alias('unique_products'),
])
```

Column Operations and Aliases

```
# alias() — rename the result of an expression
pl.col('salary').mean().alias('mean_salary')

# Operate on multiple columns at once
pl.col(['sales', 'units']).sum()          # Sum of both columns

# Map a function across a column
pl.col('name').map_elements(lambda x: x.split()[0],
return_dtype=pl.Utf8).alias('first_name')

# Null-safe operations
pl.col('salary').fill_null(strategy='mean')     # Fill nulls with column mean
pl.col('salary').fill_null(pl.col('salary').median())

# Sorting within expression
pl.col('sales').sort_by(pl.col('date'))
```

11.3 Data I/O ★ Core Training Topic

Polars provides fast, memory-efficient I/O for all major data formats. Every eager read function has a lazy scan counterpart. Understanding when to use each is critical for production data pipelines.

11.3.1 Reading CSV

pl.read_csv() — Eager (loads entire file)

```
import polars as pl

# Basic read
df = pl.read_csv('sales.csv')

# Key parameters
df = pl.read_csv(
    'sales.csv',
    separator=',',               # Delimiter (default is ',')
    has_header=True,             # Whether first row is header
    new_columns=['id','product','amount','date'],  # Override column names
    dtypes={'id': pl.Int32, 'amount': pl.Float64},  # Force dtypes
    null_values=['N/A', 'NULL', '--'],              # Strings to treat as null
    n_rows=10000,                # Read only first N rows
    skip_rows=2,                 # Skip header rows before data
    try_parse_dates=True,        # Auto-detect and parse date columns
    encoding='utf8',             # Encoding (utf8, latin1)
    comment_prefix='#',         # Skip lines starting with this char
    infer_schema_length=1000,   # Rows used to infer dtypes (default 100)
)

# Inspect result
print(df.schema)      # Column name → dtype mapping
print(df.shape)       # (rows, cols)
df.head(5)
```

pl.scan_csv() — Lazy (best for large files)

```
# scan_csv creates a LazyFrame — nothing is read yet
lf = pl.scan_csv('big_sales.csv', try_parse_dates=True)

# Build a query (still not executed)
query = (
    lf
    .filter(pl.col('amount') > 500)           # Predicate pushdown
    .select(['id', 'amount', 'region', 'date'])  # Projection pushdown
    .with_columns(
        (pl.col('amount') * 1.1).alias('amount_gst')
    )
)

# Execute the optimized query
result = query.collect()                      # Executes NOW

# Inspect query plan (what Polars will do)
print(query.explain(optimized=True))
```

Predicate pushdown means Polars pushes your .filter() condition into the CSV reader — it will skip entire row batches that don't match, reading far less data from disk.

⚠ *n_rows in scan_csv only limits rows AFTER filtering. Use it for sampling but be aware of this distinction from Pandas.*

11.3.2 Reading JSON

```
# Standard JSON (array of objects or object of arrays)
df = pl.read_json('data.json')

# Newline-Delimited JSON (NDJSON / JSON Lines) — one JSON object per line
# This format is ideal for streaming/logging because each line is independent
df = pl.read_ndjson('events.ndjson')

# Example NDJSON file content:
# {"id":1, "event":"click", "ts":"2024-01-01T10:00:00"}
# {"id":2, "event":"view",  "ts":"2024-01-01T10:00:01"}

# Lazy scan for large NDJSON files
lf = pl.scan_ndjson('large_events.ndjson')
result = lf.filter(pl.col('event') == 'purchase').collect()

# Read from JSON string
import io
json_str = '[{"name":"Alice","score":90},{"name":"Bob","score":85}]'
df = pl.read_json(io.StringIO(json_str))
```

📝 *NDJSON (newline-delimited JSON) is far more memory-efficient for large datasets than a single JSON array, because each line can be parsed independently. Use pl.scan_ndjson() for files > 100 MB.*

11.3.3 Reading Parquet — Columnar Storage Deep Dive

Why Parquet?

Parquet is a columnar binary file format designed for analytics. Key advantages:

- Columnar storage — reading only the columns you need skips entire column groups on disk
- Compressed per-column — each column type compresses efficiently (e.g., string columns with many repeats compress extremely well)
- Schema embedded — the file stores its own schema, no inference needed
- Splittable — large Parquet files can be split and processed in parallel
- Widely supported — Spark, DuckDB, BigQuery, Snowflake, Pandas all read/write it

Reading Parquet with Polars

```
import polars as pl

# Eager read
df = pl.read_parquet('data.parquet')

# Projection pushdown — only load needed columns from file
df = pl.read_parquet('data.parquet', columns=['id', 'amount', 'date'])

# Read a directory of parquet files (common in partitioned datasets)
df = pl.read_parquet('data/year=2024/**/*.parquet')

# Lazy scan — best for large files
lf = pl.scan_parquet('data.parquet')
result = (
```

```
    lf
    .filter(pl.col('date').dt.year() == 2024)
    .select(['id', 'amount', 'region'])
    .collect()
)

# Inspect schema without reading data
schema = pl.read_parquet_schema('data.parquet')
print(schema)    # {'id': Int64, 'amount': Float64, ...}
```

Compression Codecs

Codec	Speed	Compression Ratio	Best For
snappy	Very fast	Moderate (~2-3x)	Default — good all-round choice
zstd	Fast	High (~4-6x)	When file size matters more than write speed
gzip	Slow	High (~4-5x)	Compatibility with older tools
lz4	Fastest	Low (~1.5-2x)	When read/write speed is critical
uncompressed	Fastest read	None (1x)	When storage is free and speed paramount

```
# Write with specific compression
df.write_parquet('output_snappy.parquet', compression='snappy')   # Default
df.write_parquet('output_zstd.parquet',   compression='zstd', compression_level=3)
df.write_parquet('output_lz4.parquet',    compression='lz4')
```

For production pipelines where data will be read many times, prefer zstd — smaller files mean less disk I/O on each read, which often makes reads faster despite decompression overhead.

11.3.4 Reading Other Formats

Avro — Row-Oriented Binary Format

```
# Avro is common in Kafka / event streaming ecosystems
# pip install 'polars[avro]'
df = pl.read_avro('events.avro')

# Write Avro
df.write_avro('output.avro')
```

IPC / Feather — Apache Arrow IPC Format

```
# IPC (Inter-Process Communication) is Arrow's native wire format
# Also called Feather v2. Fastest read/write — no conversion overhead
df = pl.read_ipc('data.feather')        # or .arrow extension

# Lazy scan
lf = pl.scan_ipc('data.feather')

# Write IPC
df.write_ipc('output.feather')

# IPC Stream format (for streaming data)
```

```
df = pl.read_ipc_stream('stream.arrow')
df.write_ipc_stream('stream_out.arrow')
```

IPC/Feather is the fastest format for intermediate storage within a pipeline — there is virtually no serialization overhead because Polars uses Arrow natively. Use it for checkpoints in multi-step ETL.

11.3.5 Writing / Exporting Data

```
import polars as pl
df = pl.DataFrame({'id':[1,2,3], 'name':['A','B','C'], 'value':[10.1, 20.2, 30.3]})

# ── CSV ──────────────────────────────────────────────────
df.write_csv('output.csv')                      # Default comma separator
df.write_csv('output.tsv', separator='\t')       # Tab-separated
df.write_csv('output.csv', null_value='NULL')    # Custom null representation
df.write_csv('output.csv.gz')                    # Auto-compressed if .gz extension

# ── JSON ─────────────────────────────────────────────────
df.write_json('output.json')                     # Compact JSON
df.write_json('output.json', pretty=True)        # Human-readable
df.write_ndjson('output.ndjson')                 # Newline-delimited (streaming-safe)

# ── Parquet ──────────────────────────────────────────────
df.write_parquet('output.parquet')                          # Default snappy
df.write_parquet('output.parquet', compression='zstd',
                 compression_level=5, row_group_size=100000)

# ── Excel ────────────────────────────────────────────────
# pip install 'polars[xlsx2csv]' and openpyxl
df.write_excel('output.xlsx', worksheet='Results')

# ── IPC / Feather ────────────────────────────────────────
df.write_ipc('output.feather')

# ── Avro ─────────────────────────────────────────────────
df.write_avro('output.avro')
```

11.3.6 Scanning Large Files — Predicate and Projection Pushdown

Pushdown optimizations are automatically applied when using lazy (scan) functions. Understanding them helps you write more efficient queries.

Optimization	What It Does	Example
Predicate Pushdown	Passes row filters to the file reader — skips reading rows that don't match	.filter(pl.col('year') == 2024)
Projection Pushdown	Only reads columns needed — skips unused columns entirely from disk	.select(['id','amount'])
Slice Pushdown	Stops reading after N rows are collected	.limit(1000)

Optimization	What It Does	Example
Common Subplan Elimination	Avoids computing the same subquery twice	Automatic in complex plans

```
# Demonstration of pushdown optimization
lf = pl.scan_parquet('huge_dataset.parquet')    # 10 GB file

# Polars will push 'region == North' AND column selection to Parquet reader
# Result: reads only matching rows + only 3 columns from disk = tiny memory footprint
result = (
    lf
    .filter(pl.col('region') == 'North')         # Predicate pushdown
    .select(['id', 'amount', 'date'])             # Projection pushdown
    .filter(pl.col('amount') > 1000)             # Second filter (still pushed down)
    .collect()
)

# Verify pushdown happened by examining the plan
print(lf.filter(pl.col('region') == 'North').explain(optimized=True))
# PARQUET SCAN will show: predicate: [(region) == (North)]
```

Pushdown works best with Parquet files because Parquet stores row group statistics (min/max per column per chunk), enabling Polars to skip entire row groups without reading them.

11.4 Data Manipulation

Polars manipulation operations are expressed using the Expression API within contexts. This section covers the most common real-world operations with detailed examples.

Filtering and Sorting with Expressions

```
import polars as pl
df = pl.read_csv('employees.csv')

# Simple filter
df.filter(pl.col('salary') > 60000)

# Multiple conditions — use & (AND), | (OR), ~ (NOT)
df.filter(
    (pl.col('salary') > 60000) & (pl.col('dept') == 'Engineering')
)

# OR condition
df.filter(
    (pl.col('dept') == 'Sales') | (pl.col('dept') == 'Marketing')
)

# is_in() — equivalent to SQL IN
df.filter(pl.col('dept').is_in(['Sales', 'Marketing', 'HR']))

# Null checks
df.filter(pl.col('salary').is_null())        # Where salary is null
df.filter(pl.col('salary').is_not_null())    # Where salary is not null

# Sorting
df.sort('salary')                            # Ascending
df.sort('salary', descending=True)           # Descending
df.sort(['dept', 'salary'], descending=[False, True])   # Multi-column sort

# Sort nulls last (default is first)
df.sort('salary', nulls_last=True)
```

Grouping and Aggregation — group_by().agg()

```
import polars as pl
df = pl.DataFrame({
    'region' : ['North','South','North','East','South','North'],
    'product': ['A','B','A','C','A','B'],
    'sales'  : [100, 200, 150, 300, 250, 180],
    'units'  : [10, 20, 15, 30, 25, 18],
})

# Single key, multiple aggregations
df.group_by('region').agg([
    pl.col('sales').sum().alias('total_sales'),
    pl.col('sales').mean().alias('avg_sales'),
    pl.col('sales').std().alias('std_sales'),
    pl.col('units').sum().alias('total_units'),
    pl.col('product').n_unique().alias('unique_products'),
    pl.len().alias('transaction_count'),         # Row count per group
])

# Multiple grouping keys
df.group_by(['region', 'product']).agg(
```

```
    pl.col('sales').sum()
)

# Maintain group order (group_by_stable)
df.group_by('region', maintain_order=True).agg(
    pl.col('sales').sum()
)

# Top N per group using group_by + sort
df.sort('sales', descending=True).group_by('region').head(2)
```

Unlike Pandas groupby, Polars group_by does NOT guarantee order of groups in the output. Use maintain_order=True or sort() afterward if order matters.

Window Functions — over() for Group-Aware Operations

Window functions compute group-level aggregates but return a result for EVERY row (like SQL's PARTITION BY). This is equivalent to Pandas groupby().transform() but much faster.

```
import polars as pl
df = pl.DataFrame({
    'region': ['North','North','South','South','East'],
    'sales' : [100, 200, 150, 300, 250],
    'date'  : ['2024-01-01','2024-01-02','2024-01-01','2024-01-02','2024-01-01'],
})

# Add group total WITHOUT collapsing rows (unlike group_by)
df.with_columns(
    pl.col('sales').sum().over('region').alias('region_total'),
)
# Each row now has the TOTAL for its region

# Multiple window expressions
df.with_columns([
    pl.col('sales').sum().over('region').alias('region_total'),
    pl.col('sales').mean().over('region').alias('region_avg'),
    pl.col('sales').rank().over('region').alias('rank_in_region'),
    (pl.col('sales') / pl.col('sales').sum().over('region')).alias('pct_of_region'),
])

# Cumulative sum within group (ordered)
df.with_columns(
    pl.col('sales').cum_sum().over('region').alias('cumsum_by_region'),
)

# Lag/lead within group
df.with_columns(
    pl.col('sales').shift(1).over('region').alias('prev_sales'),
)
```

over() is one of Polars' most powerful features. It handles in one step what requires a merge back to the original DataFrame in Pandas. Use it for percentage-of-total, rank within group, cumulative sums per group, etc.

String Operations — .str Accessor

```
import polars as pl
df = pl.DataFrame({'name': ['Alice Smith', 'BOB JONES', ' charlie brown '],
                   'email': ['alice@gmail.com', 'bob@work.net', 'charlie@gmail.com']})

# Basic string operations
df.with_columns([
    pl.col('name').str.to_lowercase().alias('name_lower'),
    pl.col('name').str.to_uppercase().alias('name_upper'),
    pl.col('name').str.strip_chars().alias('name_trimmed'),
    pl.col('name').str.len_chars().alias('name_length'),
])

# Contains and pattern matching
df.filter(pl.col('email').str.contains('@gmail'))
df.filter(pl.col('name').str.starts_with('A'))
df.filter(pl.col('name').str.ends_with('th'))

# Split and extract
df.with_columns(
    pl.col('name').str.split(' ').alias('name_parts')       # List of parts
)
df.with_columns(
    pl.col('name').str.split_exact(' ', 1).alias('parts')   # Split into struct
)

# Replace
df.with_columns(
    pl.col('name').str.replace('BOB', 'Robert').alias('name_fixed')
)

# Regex extract
df.with_columns(
    pl.col('email').str.extract(r'([^@]+)@', group_index=1).alias('username')
)

# Slice
df.with_columns(
    pl.col('name').str.slice(0, 5).alias('name_short')   # First 5 chars
)
```

Datetime Operations — .dt Accessor

```
import polars as pl
df = pl.DataFrame({
    'ts': ['2024-01-15 09:30:00', '2024-03-22 14:45:00', '2024-07-04 00:00:00'],
}).with_columns(
    pl.col('ts').str.to_datetime('%Y-%m-%d %H:%M:%S').alias('timestamp')
)

# Extract components
df.with_columns([
    pl.col('timestamp').dt.year().alias('year'),
    pl.col('timestamp').dt.month().alias('month'),
    pl.col('timestamp').dt.day().alias('day'),
    pl.col('timestamp').dt.hour().alias('hour'),
    pl.col('timestamp').dt.weekday().alias('weekday'),    # 0=Mon, 6=Sun
    pl.col('timestamp').dt.ordinal_day().alias('day_of_year'),
    pl.col('timestamp').dt.quarter().alias('quarter'),
])
```

```
# Format as string
df.with_columns(
    pl.col('timestamp').dt.strftime('%B %Y').alias('month_year')   # e.g., 'January 2024'
)

# Date arithmetic
from datetime import timedelta
df.with_columns(
    (pl.col('timestamp') + pl.duration(days=7)).alias('next_week')
)

# Truncate to period (useful for grouping)
df.with_columns(
    pl.col('timestamp').dt.truncate('1mo').alias('month_start')    # First of month
)

# Timezone
df.with_columns(
    pl.col('timestamp').dt.replace_time_zone('UTC')
    .dt.convert_time_zone('Asia/Kolkata').alias('ist_time')
)
```

11.5 Data Cleaning

Polars distinguishes between two kinds of missing data: null (the concept of a missing/unknown value, similar to NULL in SQL) and NaN (Not a Number — a floating-point special value for undefined math like 0/0). Handling them correctly is important.

Handling Missing Values — Nulls vs NaN

Concept	Polars Value	Applies To	Detection
Missing/unknown	null	All dtypes (Int, Str, Date, etc.)	.is_null()
Math undefined	NaN	Float columns only (IEEE 754)	.is_nan()

```
import polars as pl
import math

df = pl.DataFrame({
    'id'     : [1, 2, 3, 4, 5],
    'score'  : [90.0, None, float('nan'), 85.0, None],  # None → null, nan → NaN
    'grade'  : ['A', 'B', None, 'A', 'C'],
})

# ── Detection ──────────────────────────────────────────
# Null detection (works for all types)
df.with_columns(pl.col('score').is_null().alias('score_is_null'))

# NaN detection (floats only)
df.with_columns(pl.col('score').is_nan().alias('score_is_nan'))

# Count nulls per column
df.null_count()         # Returns a 1-row DataFrame of null counts

# Percentage null
df.null_count() / df.height * 100

# ── Dropping nulls ─────────────────────────────────────
df.drop_nulls()                         # Drop rows with ANY null
df.drop_nulls(subset=['grade'])         # Drop rows where grade is null only

# ── Filling nulls ──────────────────────────────────────
df.with_columns(
    pl.col('score').fill_null(0.0).alias('score_filled'),       # Fill with literal
)
df.with_columns(
    pl.col('score').fill_null(strategy='mean').alias('score_mean_filled'),
)
df.with_columns(
    pl.col('score').fill_null(strategy='forward').alias('score_ffill'),
)
df.with_columns(
    pl.col('score').fill_null(strategy='backward').alias('score_bfill'),
)

# Fill null with another column's value
df.with_columns(
    pl.col('score').fill_null(pl.col('score').median()).alias('score_filled'),
)

# ── Filling NaN (floats only) ──────────────────────────
df.with_columns(
```

```
    pl.col('score').fill_nan(0.0).alias('score_nan_filled'),
)

# Fill both null and NaN at once
df.with_columns(
    pl.col('score').fill_nan(None).fill_null(0.0)    # Convert NaN to null first
)
```

⚠ *Do NOT confuse fill_null() with fill_nan(). Using fill_nan() on a null will not work and vice versa. When in doubt, convert NaN to null first with .fill_nan(None) then handle with fill_null().*

Detecting Outliers Using Expressions

```
import polars as pl
df = pl.read_csv('sales.csv')

# ── Z-score method ──────────────────────────────────────
df.with_columns(
    ((pl.col('amount') - pl.col('amount').mean()) / pl.col('amount').std())
    .alias('z_score')
).filter(pl.col('z_score').abs() > 3)    # Rows where |z| > 3 are outliers

# ── IQR method (robust to extreme outliers) ─────────────
q1 = df['amount'].quantile(0.25)
q3 = df['amount'].quantile(0.75)
iqr = q3 - q1

lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

# Mark outliers
df.with_columns(
    pl.when((pl.col('amount') < lower) | (pl.col('amount') > upper))
    .then(pl.lit(True))
    .otherwise(pl.lit(False))
    .alias('is_outlier')
)

# Remove outliers
df_clean = df.filter(
    pl.col('amount').is_between(lower, upper)
)
```

Data Type Casting — cast()

```
import polars as pl
df = pl.DataFrame({
    'id'     : ['1', '2', '3'],       # String, should be int
    'price'  : ['10.5', 'bad', '30'], # String, should be float
    'flag'   : [1, 0, 1],             # Int, should be bool
})

# Strict casting — raises error if conversion fails
df.with_columns(
    pl.col('id').cast(pl.Int64),       # '1' → 1  (works)
    pl.col('flag').cast(pl.Boolean),   # 1 → True, 0 → False
)

# Non-strict (permissive) casting — bad values become null
df.with_columns(
    pl.col('price').cast(pl.Float64, strict=False)   # 'bad' → null
```

```
)

# Polars data types reference
# pl.Int8, Int16, Int32, Int64     — signed integers
# pl.UInt8, UInt16, UInt32, UInt64 — unsigned integers
# pl.Float32, Float64              — floating point
# pl.Boolean                       — true/false
# pl.Utf8 / pl.String              — text
# pl.Date                          — date only (no time)
# pl.Datetime                      — date + time
# pl.Duration                      — time delta
# pl.Categorical                   — low-cardinality strings
# pl.List(pl.Int64)               — list of integers
# pl.Struct                        — struct/record type

# Convert string date column
df.with_columns(
    pl.col('date_str').str.to_date('%Y-%m-%d').alias('date'),
)

# Memory-efficient categoricals (like Pandas category dtype)
df.with_columns(
    pl.col('region').cast(pl.Categorical).alias('region_cat'),
)
```

Use pl.Categorical for columns with low cardinality (few unique values, repeated often — like 'region', 'status', 'grade'). It dramatically reduces memory and speeds up group_by operations on those columns.

11.6 Joining and Combining

Polars provides all standard SQL join types plus two additional set-operation joins (semi and anti) that are extremely useful for filtering without adding extra columns.

All Join Types

Join Type	Description	Use Case
inner	Only rows with matching keys in BOTH frames	Strict matching required
left	All rows from left, matched from right (NaN if no match)	Keep all left rows
right	All rows from right, matched from left	Keep all right rows
full / outer	All rows from both frames	Preserve all data
cross	Cartesian product — every left row x every right row	Generating combinations
semi	Left rows where a match EXISTS in right (no right columns added)	Filter existence check
anti	Left rows where NO match exists in right (no right columns added)	Find unmatched records

```
import polars as pl

orders = pl.DataFrame({
    'order_id'   : [1, 2, 3, 4],
    'customer_id': [101, 102, 103, 999],
    'amount'     : [500, 300, 700, 150],
})

customers = pl.DataFrame({
    'customer_id': [101, 102, 104],
    'name'       : ['Alice', 'Bob', 'Charlie'],
    'city'       : ['Delhi', 'Mumbai', 'Pune'],
})

# INNER JOIN — only matching customer_ids
orders.join(customers, on='customer_id', how='inner')
# Result: order_ids 1, 2 only (103 and 999 have no matching customer)

# LEFT JOIN — keep all orders
orders.join(customers, on='customer_id', how='left')
# Result: all 4 orders; order_ids 3 and 4 have null name/city

# FULL / OUTER JOIN
orders.join(customers, on='customer_id', how='full')

# SEMI JOIN — which orders have a known customer? (no extra columns)
orders.join(customers, on='customer_id', how='semi')
# Returns: order_ids 1 and 2 only

# ANTI JOIN — which orders have NO matching customer?
orders.join(customers, on='customer_id', how='anti')
# Returns: order_ids 3 and 4
```

```
# Joining on different column names
orders.join(customers, left_on='customer_id', right_on='customer_id', how='inner')

# Joining on multiple keys
df1.join(df2, on=['date', 'product_id'], how='inner')

# Handling duplicate column names with suffix
df1.join(df2, on='id', how='left', suffix='_right')
```

pl.concat() — Vertical and Horizontal Concatenation

```
import polars as pl

df_jan = pl.DataFrame({'date':['2024-01-01','2024-01-02'], 'sales':[100,200]})
df_feb = pl.DataFrame({'date':['2024-02-01','2024-02-02'], 'sales':[150,250]})

# Vertical concat (stack rows) — same columns required
combined = pl.concat([df_jan, df_feb])   # Default: vertical

# Combine many monthly files from a list
import glob
files = glob.glob('data/sales_*.csv')
all_months = pl.concat([pl.read_csv(f) for f in files])

# Horizontal concat (stack columns side by side)
# Frames must have same number of rows and no overlapping column names
df_info   = pl.DataFrame({'id':[1,2,3], 'name':['A','B','C']})
df_scores = pl.DataFrame({'math':[90,85,78], 'sci':[88,92,80]})
combined_h = pl.concat([df_info, df_scores], how='horizontal')

# Diagonal concat (handle mismatched columns — fills with null)
df_v1 = pl.DataFrame({'a':[1,2], 'b':[3,4]})
df_v2 = pl.DataFrame({'a':[5,6], 'c':[7,8]})    # Different columns
pl.concat([df_v1, df_v2], how='diagonal')        # b=null for df_v2 rows, c=null for
df_v1 rows
```

Performance Considerations — Sort-Merge vs Hash Joins

Strategy	How It Works	Best When	Polars Behavior
Hash Join	Build hash table from smaller frame, probe with larger frame	General purpose, unsorted data	Default for most joins
Sort-Merge Join	Sort both frames on join key, merge in single pass	Data is already sorted, large-large joins	Can hint with .sort() before join

```
# Polars automatically chooses the best join strategy
# For very large joins where both frames are sorted, sorting first helps

result = (
    left_df.sort('customer_id')
    .join(
        right_df.sort('customer_id'),
        on='customer_id',
        how='inner'
    )
)
```

```
# For lazy joins — Polars optimizer handles strategy selection
result = (
    pl.scan_parquet('orders.parquet')
    .join(
        pl.scan_parquet('customers.parquet'),
        on='customer_id',
        how='left'
    )
    .collect()    # Optimizer chooses best strategy here
)
```

⚠ *Cross joins (how='cross') produce N * M rows. If both frames have 1000 rows each, the result has 1,000,000 rows. Use with extreme caution on large DataFrames.*

11.7 Lab: Real-World Dataset — 1M+ Rows

This lab simulates a real e-commerce transaction pipeline. You will generate a 1M+ row dataset, process it with Polars using lazy evaluation, compare performance with Pandas, and export results.

Setup — Generate 1M Row Dataset

```
import polars as pl
import pandas as pd
import numpy as np
import time, random

# ── Generate synthetic 1M row dataset ──────────────────────
np.random.seed(42)
N = 1_000_000

regions   = ['North', 'South', 'East', 'West', 'Central']
products  = ['Electronics','Clothing','Food','Books','Sports','Toys']
statuses  = ['Completed','Pending','Cancelled','Refunded']

data = {
    'transaction_id': range(1, N+1),
    'customer_id'   : np.random.randint(10000, 99999, N),
    'region'        : np.random.choice(regions,  N),
    'product'       : np.random.choice(products, N),
    'amount'        : np.round(np.random.exponential(250, N), 2),
    'units'         : np.random.randint(1, 20, N),
    'status'        : np.random.choice(statuses, N, p=[0.7,0.15,0.1,0.05]),
    'date'          : pd.date_range('2022-01-01', periods=N, freq='1min').astype(str),
}

# Save as CSV and Parquet for lab use
df_pandas = pd.DataFrame(data)
df_pandas.to_csv('transactions.csv', index=False)
df_pandas.to_parquet('transactions.parquet', index=False)
print(f'Dataset created: {N:,} rows')
```

Step 1 — Load and Inspect with Polars (Lazy)

```
import polars as pl, time

# Use lazy scan for memory efficiency
lf = pl.scan_parquet('transactions.parquet')

# Inspect schema without loading data
print(lf.schema)

# Quick row count (lazy — reads only metadata)
print(lf.select(pl.len()).collect())

# Preview without loading everything
print(lf.limit(5).collect())
```

Step 2 — Clean Data

```
# Build cleaning pipeline (all lazy)
lf_clean = (
    lf
```

```
    # Parse date string to datetime
    .with_columns(
        pl.col('date').str.to_datetime('%Y-%m-%d %H:%M:%S').alias('timestamp')
    )
    # Remove cancelled transactions
    .filter(pl.col('status') != 'Cancelled')
    # Remove negative amounts (data quality)
    .filter(pl.col('amount') > 0)
    # Cast region and product to Categorical
    .with_columns([
        pl.col('region').cast(pl.Categorical),
        pl.col('product').cast(pl.Categorical),
    ])
    # Add derived time columns
    .with_columns([
        pl.col('timestamp').dt.year().alias('year'),
        pl.col('timestamp').dt.month().alias('month'),
        pl.col('timestamp').dt.quarter().alias('quarter'),
        (pl.col('amount') * pl.col('units')).alias('total_value'),
    ])
)

# Execute cleaning pipeline
t0 = time.time()
df_clean = lf_clean.collect()
print(f'Polars clean: {time.time()-t0:.3f}s  |  Rows: {df_clean.height:,}')
```

Step 3 — Complex Transformations

```
# ── GroupBy Aggregation ──────────────────────────────────
t0 = time.time()
monthly_summary = (
    df_clean
    .group_by(['year', 'month', 'region', 'product'])
    .agg([
        pl.col('total_value').sum().alias('revenue'),
        pl.col('total_value').mean().alias('avg_order_value'),
        pl.col('customer_id').n_unique().alias('unique_customers'),
        pl.len().alias('transaction_count'),
    ])
    .sort(['year', 'month', 'revenue'], descending=[False, False, True])
)
print(f'GroupBy: {time.time()-t0:.3f}s')

# ── Window Function — Region contribution % ──────────────
t0 = time.time()
df_with_pct = df_clean.with_columns([
    pl.col('total_value').sum().over('region').alias('region_total'),
    (pl.col('total_value') / pl.col('total_value').sum().over('region') * 100)
    .round(2).alias('pct_of_region'),
    pl.col('total_value').rank(descending=True).over('region').alias('rank_in_region'),
])
print(f'Window functions: {time.time()-t0:.3f}s')

# ── String Operations ────────────────────────────────────
t0 = time.time()
df_strings = df_clean.with_columns([
    pl.col('product').cast(pl.Utf8).str.to_uppercase().alias('product_upper'),
    pl.col('region').cast(pl.Utf8).str.to_lowercase().alias('region_lower'),
    pl.col('status').str.len_chars().alias('status_len'),
])
print(f'String ops: {time.time()-t0:.3f}s')
```

Step 4 — Performance Comparison with Pandas

```
import pandas as pd, time

df_pd = pd.read_parquet('transactions.parquet')

# ── Test 1: GroupBy ──────────────────────────────────────
# Pandas
t0 = time.time()
result_pd = df_pd.groupby(['region','product'])['amount'].agg(['sum','mean','count'])
time_pd = time.time() - t0

# Polars
t0 = time.time()
result_pl = (
    pl.scan_parquet('transactions.parquet')
    .group_by(['region','product'])
    .agg([pl.col('amount').sum(), pl.col('amount').mean(), pl.len()])
    .collect()
)
time_pl = time.time() - t0

print(f'GroupBy — Pandas: {time_pd:.3f}s | Polars: {time_pl:.3f}s | Speedup:
{time_pd/time_pl:.1f}x')

# ── Test 2: Filter + Sort ────────────────────────────────
# Pandas
t0 = time.time()
r_pd = df_pd[df_pd['amount'] > 500].sort_values('amount', ascending=False).head(1000)
time_pd = time.time() - t0

# Polars (lazy)
t0 = time.time()
r_pl = (
    pl.scan_parquet('transactions.parquet')
    .filter(pl.col('amount') > 500)
    .sort('amount', descending=True)
    .limit(1000)
    .collect()
)
time_pl = time.time() - t0

print(f'Filter+Sort — Pandas: {time_pd:.3f}s | Polars: {time_pl:.3f}s | Speedup:
{time_pd/time_pl:.1f}x')

# ── Test 3: String Operations ────────────────────────────
# Pandas
t0 = time.time()
r_pd = df_pd[df_pd['region'].str.startswith('N')]
time_pd = time.time() - t0

# Polars
t0 = time.time()
r_pl = pl.scan_parquet('transactions.parquet').filter(
    pl.col('region').str.starts_with('N')).collect()
time_pl = time.time() - t0

print(f'String filter — Pandas: {time_pd:.3f}s | Polars: {time_pl:.3f}s | Speedup:
{time_pd/time_pl:.1f}x')
```

Step 5 — Export Results in Multiple Formats

```
# Export monthly summary
monthly_summary.write_parquet('monthly_summary.parquet', compression='zstd')
monthly_summary.write_csv('monthly_summary.csv')
monthly_summary.write_excel('monthly_summary.xlsx', worksheet='Monthly')

# Export cleaned dataset
df_clean.write_parquet('transactions_clean.parquet', compression='snappy')

print('All exports complete!')
print(f'Cleaned dataset: {df_clean.height:,} rows x {df_clean.width} columns')

# Memory usage comparison
import sys
print(f'Polars memory: {df_clean.estimated_size("mb"):.1f} MB')
print(f'Pandas memory: {df_pd.memory_usage(deep=True).sum()/1e6:.1f} MB')
```

Jupyter Notebook Timing with %%timeit

```
# In Jupyter, use %%timeit for accurate multi-run benchmarks

# Cell 1 — Pandas
%%timeit
df_pd.groupby('region')['amount'].sum()

# Cell 2 — Polars eager
%%timeit
df_pl.group_by('region').agg(pl.col('amount').sum())

# Cell 3 — Polars lazy (from file)
%%timeit
pl.scan_parquet('transactions.parquet').group_by('region').agg(pl.col('amount').sum()).co
llect()

# %%timeit runs the code multiple times and reports mean +/- std
# Use %timeit for single-line timing
%timeit df_pl.sort('amount')
```

In production, always measure with your ACTUAL data. Microbenchmarks can be misleading. The Polars advantage grows with: larger data, more complex operations, and I/O-heavy workflows.

Quick Reference — Polars Cheat Sheet

I/O

Operation	Code
Read CSV (eager)	pl.read_csv('file.csv', separator=',', try_parse_dates=True)
Read CSV (lazy)	pl.scan_csv('file.csv')
Read Parquet (eager)	pl.read_parquet('file.parquet', columns=['col1','col2'])
Read Parquet (lazy)	pl.scan_parquet('file.parquet')
Read JSON	pl.read_json('file.json')
Read NDJSON (lazy)	pl.scan_ndjson('file.ndjson')
Write CSV	df.write_csv('out.csv')
Write Parquet	df.write_parquet('out.parquet', compression='zstd')
Write Excel	df.write_excel('out.xlsx', worksheet='Sheet1')
Execute lazy query	lf.collect()

Selection & Filtering

Task	Code
Select columns	df.select(['col1','col2'])
Add computed col	df.with_columns((pl.col('a')+pl.col('b')).alias('c'))
Filter rows	df.filter(pl.col('amount') > 100)
Filter with AND	df.filter((pl.col('a')>5) & (pl.col('b')<10))
Filter with isin	df.filter(pl.col('region').is_in(['North','South']))
Null filter	df.filter(pl.col('col').is_not_null())
Conditional column	pl.when(cond).then(val1).otherwise(val2).alias('name')
Sort ascending	df.sort('col')
Sort descending	df.sort('col', descending=True)

Grouping & Aggregation

Task	Code
GroupBy sum	df.group_by('region').agg(pl.col('sales').sum())
Multiple aggs	df.group_by('r').agg([pl.col('s').sum(), pl.col('s').mean()])
Window function	df.with_columns(pl.col('sales').sum().over('region').alias('total'))
Rank in group	df.with_columns(pl.col('sales').rank().over('region'))
Cumsum in group	df.with_columns(pl.col('sales').cum_sum().over('region'))

Cleaning & Types

Task	Code
Null count	df.null_count()
Drop null rows	df.drop_nulls() / df.drop_nulls(subset=['col'])
Fill null (literal)	df.with_columns(pl.col('c').fill_null(0))
Fill null (strategy)	df.with_columns(pl.col('c').fill_null(strategy='mean'))
Fill NaN	df.with_columns(pl.col('c').fill_nan(0.0))
Cast type	df.with_columns(pl.col('id').cast(pl.Int32))
Permissive cast	df.with_columns(pl.col('x').cast(pl.Float64, strict=False))
To categorical	df.with_columns(pl.col('region').cast(pl.Categorical))

Joins & Concat

Task	Code
Inner join	df1.join(df2, on='id', how='inner')
Left join	df1.join(df2, on='id', how='left')
Semi join (filter)	df1.join(df2, on='id', how='semi')
Anti join (exclude)	df1.join(df2, on='id', how='anti')
Vertical concat	pl.concat([df1, df2])
Horizontal concat	pl.concat([df1, df2], how='horizontal')
Diagonal concat	pl.concat([df1, df2], how='diagonal')

String & DateTime

Task	Code
Lowercase	pl.col('s').str.to_lowercase()
Contains	pl.col('s').str.contains('pattern')
Extract regex	pl.col('s').str.extract(r'(\w+)', group_index=1)
Split	pl.col('s').str.split(' ')
Parse datetime	pl.col('s').str.to_datetime('%Y-%m-%d')
Extract year	pl.col('ts').dt.year()
Extract month	pl.col('ts').dt.month()
Format datetime	pl.col('ts').dt.strftime('%B %Y')
Truncate to month	pl.col('ts').dt.truncate('1mo')

Polars vs Pandas — Key Differences

Feature	Pandas	Polars
Row index	Has index (can be custom)	No index — position only

Feature	Pandas	Polars
Column selection	df['col'] / df.col	pl.col('col') in contexts
Null type	NaN for all types	null (all types) + NaN (float)
GroupBy result order	Ordered by key	Unordered (use maintain_order=True)
Lazy eval	No (use Dask)	Built-in (LazyFrame)
Slice end in .loc	Inclusive	Not applicable (no .loc)
applymap equiv	.map_elements() on Series	Same — .map_elements()
category dtype	pd.Categorical	pl.Categorical
Transformation	groupby().transform()	.over() in with_columns()

Vikas Karanth U at 2/25/2026 8:51 PM